LANCE: An LLM Agent for Network Compromise Evaluation with IoTChainBench

Tanguy Vansnick*, Gaspard Menou†, Saïd Mahmoudi*
*University of Mons, Mons, Belgium
†Centrale Marseille, Marseille, France
tanguy.vansnick@umons.ac.be

***Abstract*—LLM penetration-testing agents are typically evaluated on isolated hosts, while IoT/OT compromises often depend on reachability across gateways, brokers, and segmented services. This leaves a methodological gap: single-host corpora do not measure pivots, protocol roles, or evidence depth. We present IoTChainBench, a reproducible network-scale IoT benchmark with 12 Proxmox/Ansible scenarios, 116 virtual devices, 209 injected vulnerabilities, and per-vulnerability ground truth across reachable and pivot-gated services. We also introduce LANCE (LLM Agent for Network Compromise Evaluation), a six-phase network-native harness for topology analysis, active discovery, triage, verification, intrusion attempts, and report generation. Under the same model and evaluator, LANCE reaches 0.935 F1 in informed mode (topology prior) and 0.887 F1 in blind mode (active discovery only), outperforming both adapted single-host baselines on all 12 scenarios. Across three additional LLM backends, LANCE remains in a narrow 0.945–0.947 mean-F1 band. On external single-host benchmarks used as transfer checks, LANCE reaches 27.3% end-to-end success on AutoPenBench and 30.2% useful findings on a 328-case Vulhub snapshot.**

***Index Terms*—IoT security, penetration testing, large language models, autonomous agents, benchmark, cyber-physical systems**

1. Introduction

IoT/OT deployments are compromised through paths, not just devices. A camera, broker, gateway, or PLC may be low impact in isolation but high impact when it enables access to a segmented service. Yet LLM penetration-testing agents are still evaluated mostly on isolated hosts, containers, or CTF-style services scored independently. Such benchmarks cannot measure whether an agent discovers pivot-gated vulnerabilities, reasons over IoT/OT protocols such as MQTT, Modbus, CoAP, or LoRaWAN, or produces evidence before reporting a finding.

Contributions. This paper makes three contributions.

- **IoTChainBench:** a reproducible network-scale IoT benchmark with 12 Proxmox/Ansible scenarios, 116 virtual devices, 209 injected vulnerabilities, firewall-enforced reachability, and per-vulnerability ground truth.
- **LANCE:** a six-phase network-native LLM harness for discovery, triage, verification, intrusion, and reporting across multi-device IoT networks, with normalized finding schemas and evidence-level scoring.
- **Evaluation:** a controlled comparison showing that LANCE reaches 0.935 mean F1 in informed mode and 0.887 in blind mode, outperforms both adapted baselines on all 12 scenarios, and remains stable across three additional LLM backends.

All artifacts are released at `ANONYMIZED_URL`.

2. Related Work

2.1. LLM Penetration-Testing Agents

LLM penetration-testing agents have evolved from single-agent shell-driving systems to multi-agent and model-specialized pipelines. Early systems such as Happe and Cito [1] and PENTESTGPT [2] showed that LLMs can plan and execute attacks on isolated VMs and CTF-style targets. Later systems, including HACKINGBUDDYGPT [3], AUTOATTACKER [4], VULNBOT [5], AUTOPENTESTER [6], CAI [7], and XOFFENSE [8], improve tool use, multi-agent planning, retrieval, or model specialization.

Despite these advances, reported evaluations remain primarily single-target or challenge-level. They do not score subnet reachability, pivot depth, or per-vulnerability progress across IoT/OT protocols such as MQTT, Modbus, LoRaWAN, and CoAP. This is the gap addressed by IoTChainBench and LANCE.

2.2. LLM Cybersecurity Benchmarks

Existing LLM cybersecurity benchmarks mainly score challenge completion. AUTOPENBENCH [9] uses 33 Docker-container tasks with milestone and flag-based scoring, while VULHUB [10], CAIBENCH [11], NYU CTF BENCH [12], and CYBENCH [13] cover vulnerable services, CTF tasks, attack-and-defense ranges, or knowledge tests. These corpora are useful for measuring exploit progress on isolated targets, but they do not provide per-vulnerability ground truth tied to reachability, hop depth, or IoT/OT protocol roles.

Multi-host cyber ranges such as LUDUS [14] and GOAD [15] model lateral movement in enterprise Active

Directory networks, but they are training labs rather than scored LLM benchmarks. They do not ship an IoT/OT protocol taxonomy, per-vulnerability oracle, or evaluator. IoTChainBench fills this gap by combining deployable multi-device IoT networks with reachability-aware ground truth and evidence-level scoring.

2.3. Attack Graphs and Network Reachability

Classical attack-graph systems model how vulnerabilities compose across a network. Early graph-based analysis [16], model-checking approaches [17], and Datalog-based systems such as MULVAL [18] compute reachable attacker states from static host and service descriptions. LANCE reuses this reachability view but applies it to live LLM-driven assessment: edges are validated by probes, actions are selected by the agent, and the graph is updated as hosts and paths are discovered.

2.4. IoT Security Context

Scenario design and ground-truth taxonomy draw on the OWASP Internet of Things project's IoT Top 10 categories [19], MITRE ATT&CK for ICS [20] for OT-tactic annotations, and NIST SP 800-82 Rev. 3 [21] for OT segmentation conventions.

Existing IoT security artifacts mainly target either traffic classification or single-device firmware analysis. Network-traffic datasets such as IoT-23 [22], TON_IoT [23], and Edge-IIoTset [24] provide labeled traces for intrusion detection, but not deployable pentest targets with controllable reachability. Firmware-focused work such as Costin *et al.* [25], FIRMADYNE [26], FIRMAE [27], and IOTGOAT [28] exercises single-device analysis. IoTChainBench instead evaluates active assessment on deployed multi-device IoT networks where firewall rules enforce which services are directly reachable and which require a pivot, with per-vulnerability ground truth.

3. Threat Model and Problem Definition

Problem. Given a target subnet and an entry point e, network-scale penetration testing requires discovering reachable hosts, identifying vulnerabilities, producing evidence of exploitability, and reporting findings across the full network. The unit of evaluation is the network rather than an individual host: some high-impact IoT/OT vulnerabilities are directly exposed, while others become reachable only through gateway or zone-transition pivots. Single-host metrics cannot capture this reachability distinction.

Attacker model. The attacker has Layer-3 access to the target subnet but no prior credentials, out-of-band software inventory, or privileged vantage point. They may use standard pentest tools (Nmap, MQTT clients, SSH, HTTP) through an LLM tool-calling agent under a bounded turn budget. Their objective is data exfiltration or access to OT services behind segmentation boundaries.

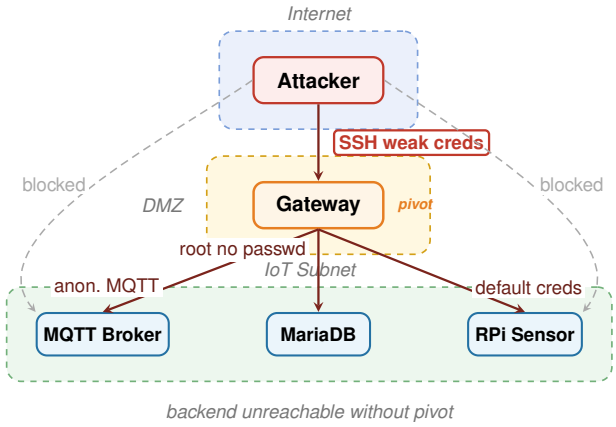

Figure 1. Gateway-centric attack chain (Scenario S3). Backend services are unreachable from the Internet without first pivoting through the gateway. Single-host agents fail on this topology (F1 = 0 for both baselines) because they scan each target independently.

Evidence levels. A finding is credited at the deepest level the agent demonstrates: L1 detection, where a scan or banner confirms an issue; L2 exploitation, where active interaction demonstrates it; and L3 exfiltration, where sensitive content is retrieved. These levels guide LANCE Phase 4 and evaluator scoring.

Scope. We evaluate IP/application-layer attacks on virtual IoT networks. Physical, side-channel, RF-layer, firmware-extraction, persistence, and insider attacks are out of scope.

Figure 1 illustrates a gateway-centric scenario where backend services are unreachable until the gateway is compromised.

4. IoTChainBench: A Network-Scale Benchmark

IoTChainBench is designed to evaluate network-scale IoT/OT assessment rather than isolated-host exploitation. Scenarios are deployed reproducibly from Ansible playbooks on Proxmox, vulnerabilities are injected idempotently, and segmented cases use OpenWrt firewall rules to enforce which services are directly reachable and which require a pivot. The benchmark covers five reachability patterns: flat networks (P1), gateway-centric deployments (P2), segmented IT/OT zones (P3), hub-centric stars (P4), and edge-cloud pivots (P5). Scenarios mix IoT/OT protocols such as MQTT, Modbus, LoRaWAN, CoAP, SNMP, SSH, HTTP, FTP, and Telnet, with per-vulnerability YAML ground truth recording device, IP, severity, category, indicators, and verification commands.

Design goals. The benchmark is built around three requirements:

- **Deployability:** every scenario is generated from Ansible and Proxmox artifacts, so runs can be recreated rather than approximated from static descriptions.

- **Reachability awareness:** firewall rules encode which services are exposed from the entry point and which become reachable only after a gateway or zone transition.
- **Scorable evidence:** the oracle records expected indicators and verification commands, allowing the evaluator to distinguish shallow detection from stronger exploit evidence.

The 12 scenarios (Table 1) comprise 10 main scenarios covering 4–8 devices and two scalability stressors with 15 and 35 devices.

TABLE 1. IOTCHAINBENCH SCENARIO CATALOGUE.

ID	Pattern / role	Dev	Vulns	Theme
Main scenarios (pattern coverage)				
`s_1`	P1 Flat	4	12	Minimal IoT (MQTT/Web/SSH)
`s_2`	P1 Flat	6	13	Flat IoT + IT mix
`s_3`	P2 Gateway-centric	8	18	NATO lab replica
`s_4`	P3 Segmented IT/OT	8	18	Industrial (Modbus/HMI)
`s_5`	P4 Hub-centric star	8	15	Smart building (cameras/NVR)
`s_6`	P4 Hub-centric star	6	16	Home automation hub
`s_7`	P5 Edge-cloud pivot	6	14	Edge $\rightarrow$ cloud API
`s_8`	P3 Segmented IT/OT	8	14	IT/IoT/OT 3-zone industrial
`s_9`	P1 Flat (mesh)	6	11	Mesh IoT smart-city outdoor
`s_10`	P1 Flat (variants)	6	13	Role-variant stress
Scalability stressors				
`s_11`	Smart City (flat)	15	23	3 zones same /24 (no isolation)
`s_12`	Smart City (large)	35	42	Largest network (34 LXC services)
Total		**116**	**209**	

4.1. Deterministic Vulnerability Injection

Vulnerabilities are injected by a dedicated Ansible role parameterized by `scenario_id`, making deployments reproducible and idempotent. Scenario YAML files declare router-level weaknesses (Telnet, WAN administration, FTP) and per-service roles that trigger targeted injections such as anonymous MQTT, weak SSH credentials, and passwordless database roots.

Across the 12 scenarios, IoTChainBench injects 209 vulnerabilities in 11 categories. The largest categories are misconfiguration (62), data exposure (36), missing authentication (34), and default credentials (26). Severity is intentionally skewed toward exploitable deployment weaknesses: 33 critical, 95 high, 60 medium, and 21 low.

4.2. Ground-Truth Schema

Each scenario ships a YAML ground-truth file with one entry per injected vulnerability. Entries record the affected device, IP, role, severity, category, OWASP IoT Top 10 and MITRE ATT&CK for ICS mappings, optional CVE, expected indicators, verification command, and reachability metadata. This per-vulnerability granularity supports the common matcher and severity-weighted score used in evaluation, while preserving pivot-aware diagnostics absent from flag-capture benchmarks.

Scenarios may also declare adjacent advisory vulnerability types that are counted as neither true positives nor false positives. This avoids penalizing legitimate hardening recommendations while keeping the recall denominator fixed.

4.3. Scoring and Diagnostic Fields

Each LLM finding is matched to ground truth in three ordered passes: exact CVE, `(ip, vuln_type)` with canonicalized type, then `(ip, severity)` fallback. Unmatched findings are false positives, and duplicates on `(ip, type, port)` are collapsed to prevent inflation.

We report recall, precision, F1, and a severity-weighted score. The latter normalizes true positives by ground-truth severity weights (critical 4, high 3, medium 2, low 1), with $0.5\times$ and $0.75\times$ penalties for fallback and severity-mismatch matches. F1 treats each vulnerability as binary, while the weighted score rewards recovering more severe findings.

For diagnostics, ground truth can store the minimum hop depth from the entry point to each affected device. We use this metadata to design multi-hop scenarios and interpret failures, but do not report hop-depth as a separate aggregate metric.

5. LANCE: An LLM Agent for Network Compromise Evaluation

Existing LLM pentest harnesses typically process targets as a flat list, letting context grow with network size. LANCE instead decomposes network assessment into six phases with typed deliverables. Its key design choice is device- and vulnerability-level parallelism: constrained micro-agents analyze individual targets or findings, and their outputs are merged deterministically into a network-level state. This keeps early-stage contexts bounded as the number of devices grows, while cross-device reasoning is deferred to the intrusion and reporting phases. LLM-produced vulnerability types are canonicalized into the benchmark taxonomy before scoring, reducing noise and duplicate findings.

5.1. Pipeline

LANCE separates network assessment into discovery, local analysis, evidence collection, lateral movement, and reporting. **Phase 1** builds a directed topology graph $G = (V, E)$ whose vertices are devices and whose edges are protocol-labeled links. In informed mode, this graph is loaded from benchmark metadata; in blind mode, it starts empty and is populated by active discovery. **Phase 2** scans the target CIDR for hosts, services, versions, L2 vendors, and protocol endpoints (MQTT, SSH, HTTP). **Phase 3** performs per-device triage: each device receives a deterministic scanner pass and a constrained LLM micro-agent, whose outputs are merged and deduplicated. **Phase 4** verifies candidate findings with operational tools such as SSH, MQTT, MySQL, Modbus, and Telnet, attempting the deepest evidence level achievable (L1–L3). **Phase 5** conducts intrusion and lateral movement over observed or asserted topology edges, recording each hop with source, destination, protocol, and credentials. **Phase 6** emits a network-scoped report with device inventory, validated findings, attack-surface summary, intrusion narrative, and remediation priorities.

5.2. Cost Envelope

For a network with n devices and v candidate findings per device on average, LANCE issues $O(nv)$ LLM invocations: four global calls (Phases 1, 2, 5, and 6), n per-device triage calls (Phase 3), and nv per-finding verification calls (Phase 4). Each invocation has a fixed turn budget (50 for Phases 1–2, 10 for Phases 3–4, 30 for Phase 5, and 20 for Phase 6), so cost scales with observed devices and findings rather than with combinations of network paths.

6. Experimental Setup

Infrastructure. All scenarios run as LXC containers on a Proxmox VE host, connected through a dedicated Linux bridge with an OpenWrt gateway. Deployment and teardown are fully automated with Ansible.

Model. For the architecture comparison, all LLM-driven systems use the same general-purpose model, **MiniMax-M2.7** [29], with fixed prompts and temperature 0 decoding. This controls for model choice while focusing the comparison on architecture rather than fine-tuning.

Baselines. We compare LANCE with two adapted LLM-agent baselines:

- a CAI-style single-agent tool loop [7];
- a VulnBot-style multi-agent pipeline with a penetration task graph [5].

Both baselines are designed primarily for single-target workflows, so we run them in the informed setting once per ground-truth target device, normalize their outputs into the common finding schema, merge findings at scenario level, and score the merged result with the same evaluator. LANCE is reported in both informed and blind modes.

This adaptation is intentionally favorable to the baselines: they receive the ground-truth target devices instead of having to discover the network from scratch. What they do not receive is the network-native state maintained by LANCE, including shared topology, pivot paths, and scenario-level deduplication before intrusion planning. The comparison therefore isolates whether architecture matters once model choice, prompts, target artifacts, and evaluator are controlled.

Metrics and variance. We report recall, precision, F1, and severity-weighted score. Evidence levels and hop-depth metadata are retained for diagnostic analysis but not reported as aggregate metrics. LANCE is additionally repeated once in informed mode to estimate same-artifact stability; adapted baselines are reported from one controlled run per target device.

7. Evaluation

7.1. Overall Performance

Table 2 reports per-scenario F1 across all four systems. Informed LANCE reaches 0.935 mean F1 (0.959 recall, 0.914 precision), while blind LANCE reaches 0.887 mean F1 without a topology prior. Both modes outperform both adapted baselines on every scenario, and the baselines never exceed F1 = 0.55. The scalability scenarios confirm the trend: `S11` (15 devices) reaches F1 = 0.875 and `S12` (35 devices) reaches F1 = 0.942 in informed mode.

TABLE 2. PER-SCENARIO F1 ON ALL 12 SCENARIOS (REFERENCE RUN).

Scenario	Informed	Blind	CAI	VulnBot
S1	0.923	0.880	0.153	0.153
S2	0.923	0.923	0.353	0.445
S3	0.947	0.941	0.000	0.000
S4	0.944	0.909	0.363	0.435
S5	1.000	0.929	0.500	0.500
S6	1.000	1.000	0.400	0.400
S7	0.965	0.889	0.500	0.445
S8	0.933	0.923	0.546	0.353
S9	0.952	0.857	0.167	0.286
S10	0.815	0.769	0.143	0.143
S11	0.875	0.857	0.414	0.414
S12	0.942	0.762	0.242	0.307
Mean	0.935	0.887	0.315	0.323

F1 = 0 means that the normalized output contained no finding matching the scenario ground truth under the common evaluator.

Beyond F1, informed LANCE reaches 86.4% severity-weighted score on average, compared with 73.8% in blind mode. Blind mode has slightly higher mean precision (0.922 vs. 0.914) but lower recall (0.856 vs. 0.959), showing that active discovery reduces some false positives while missing vulnerabilities that the topology prior helps expose or prioritize. The clearest multi-hop failure case is `S3`: both adapted baselines produce no true-positive matches, while LANCE remains above F1 = 0.94 in both modes.

Per-scenario interpretation. The scenario spread helps explain the aggregate result. `S3` isolates pivot reasoning: the vulnerable back-end services are present but unreachable until the gateway is traversed, so flat per-target baselines fail even when run with target knowledge. `S10` is the lowest informed LANCE score (F1 = 0.815), indicating that role variants and naming heterogeneity still create matching and prioritization errors. `S12` stresses network size: informed LANCE remains high on 35 devices (F1 = 0.942), while blind mode drops to 0.762, showing that active discovery remains useful but topology metadata becomes more valuable as the device set grows. These cases motivate reporting per-scenario scores rather than only macro averages.

7.2. Rerun Consistency

To estimate run-to-run stability, informed LANCE was executed twice on the same artifact set. The second run reaches mean F1 = 0.918 versus 0.935 for the reference run ($\Delta = -0.017$). Seven scenarios vary by less than 0.05 F1; the largest drops occur on `S7` (-0.182) and `S4` (-0.156), while `S10` improves by $+0.145$. Blind mode was executed once per scenario, so full blind-mode confidence intervals remain future work.

7.3. Model Sensitivity

To assess whether LANCE depends on a single LLM backend, we rerun the same architecture with MiniMax-M2.7, DeepSeek V4 Flash, Gemma 4 12B, and Qwen3-14B under identical prompts, tool budgets, artifacts, and evaluator. Table 3 reports macro-averaged results over the same 12 scenarios. The three additional backends remain close to the MiniMax reference: F1 ranges from 0.945 to 0.947, while severity-weighted score ranges from 83.0% to 85.5%.

TABLE 3. MODEL SENSITIVITY OF LANCE ON IOTCHAINBENCH.

Model	Recall	Precision	F1	Sev.-weighted
MiniMax-M2.7	0.959	0.914	0.935	86.4%
DeepSeek V4 Flash	0.933	0.964	0.947	85.5%
Gemma 4 12B	0.917	0.976	0.945	83.0%
Qwen3-14B	0.923	0.976	0.947	84.1%

The main cross-model variation is a precision–recall tradeoff rather than a collapse in task performance: Gemma and Qwen are more conservative, while DeepSeek obtains the highest severity-weighted score among the additional backends.

7.4. Single-Host Transfer Checks

Although the main claim concerns network-scale IoT evaluation, we also run LANCE on two established single-host corpora to check that the per-target exploitation component is not specialized only to IoTChainBench. These runs are contextual rather than a controlled leaderboard because task oracles and published protocols differ across systems. On AutoPenBench, informed LANCE captures 9/33 end-to-end flags (27.3%) and produces useful exploit evidence on 17/33 tasks (51.5%); blind mode captures 7/33 flags (21.2%). On a 328-case Vulhub snapshot, informed LANCE produces 99 useful findings (30.2%), while blind mode produces 92 (28.0%). These results support transfer to single-host exploitation, but they do not replace the main IoTChainBench result because they do not exercise topology priors, reachability constraints, or lateral movement.

8. Discussion and Limitations

The evaluation suggests three implications.

- **Flat target lists are insufficient.** The clearest case is `S3`: back-end services are unreachable without a prior gateway traversal step. Both adapted baselines produce no true-positive matches (F1 = 0), while LANCE exceeds F1 = 0.94 in both informed and blind modes.
- **Blind discovery is viable but changes the error profile.** Blind LANCE keeps high precision and F1, but loses recall relative to informed mode. This suggests that active discovery can recover much of the topology, while metadata still helps expose or prioritize pivot-gated findings.
- **The result is architectural, not only model-specific.** The architecture comparison uses the same model and evaluator, and the Gemma, DeepSeek, and Qwen reruns remain within a narrow F1 band under the same harness.

Several limitations remain.

- **Virtualization boundary:** scenarios run on Linux containers; real constrained hardware, bootloaders, RTOS firmware, and physical interfaces introduce attack surfaces not covered here.
- **Run-to-run variance:** only informed LANCE was rerun on the same artifact set (ΔF1 = -0.017 mean); blind-mode confidence intervals remain future work.
- **Oracle boundaries:** neutral advisory categories may inflate precision if they are defined too broadly, and severity-fallback matches are only an approximation of evidence quality.
- **Prompt and model scope:** results use a single prompt family; per-model prompt tuning may change absolute scores even if the architecture trend persists.
- **Scenario coverage:** the benchmark covers fixed topological patterns and vulnerabilities tractably injectable via Ansible; dynamic deployments, RF-layer attacks, and supply-chain weaknesses are out of scope.

9. Conclusion

Single-host benchmarks cannot capture the pivot chains, protocol roles, and reachability dependencies that shape IoT/OT risk. This paper introduced IoTChainBench, a 12-scenario network-scale benchmark with deterministic vulnerability injection and per-vulnerability ground truth, and LANCE, a six-phase network-native harness for discovery, verification, intrusion, and reporting.

The evaluation supports three conclusions.

- **Network context matters:** informed LANCE reaches 0.935 mean F1 and 86.4% severity-weighted score, outperforming both adapted single-target baselines on every scenario.
- **Topology priors help but are not mandatory:** blind LANCE reaches 0.887 mean F1 using active discovery only, with lower recall but slightly higher precision than informed mode.
- **The effect is not model-specific:** Gemma, DeepSeek, and Qwen reruns remain within 0.945–0.947 mean F1 under the same architecture and evaluator.

Together, these results show that evaluating LLM-based IoT penetration-testing agents requires network-aware tasks, per-vulnerability ground truth, and evidence-level scoring. We release the scenarios, playbooks, ground truth, and evaluation code to support reproducible research in this setting.

References

[1] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," in *Proc. ACM ESEC/FSE*, 2023.

[2] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Security Symp. (USENIX Security 24)*, 2024, pp. 847–864. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/deng

[3] A. Happe *et al.*, "HackingBuddyGPT: An LLM-driven pentest agent," 2024. [Online]. Available: https://github.com/ipa-lab/hackingBuddyGPT

[4] J. Xu *et al.*, "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," *arXiv:2403.01038*, 2024.

[5] H. Kong *et al.*, "VulnBot: Autonomous penetration testing for a multi-agent collaborative framework," *arXiv:2501.13411*, 2025. [Online]. Available: https://arxiv.org/abs/2501.13411

[6] Y. Ginige *et al.*, "AutoPentester: An LLM agent-based framework for automated pentesting," *arXiv:2510.05605*, 2025.

[7] V. Mayoral-Vilches *et al.*, "CAI: An open, bug bounty-ready cybersecurity AI," *arXiv:2504.06017*, 2025.

[8] P. D. Luong *et al.*, "xOffense: An autonomous multi-agent framework for penetration testing with domain-adapted large language models," *arXiv:2509.13021*, 2025.

[9] L. Gioacchini, M. Mellia, I. Drago, A. Delsanto, G. Siracusano, and R. Bifulco, "AutoPenBench: Benchmarking generative agents for penetration testing," *arXiv:2410.03225*, 2024.

[10] Vulhub contributors, "Vulhub: Pre-built vulnerable Docker environments for learning and reproducing CVEs." [Online]. Available: https://github.com/vulhub/vulhub

[11] M. Sanz-Gómez, V. Mayoral-Vilches, F. Balassone, L. J. Navarrete-Lozano, C. R. J. Veas Chavez, and M. del Mundo de Torres, "Cybersecurity AI Benchmark (CAIBench): A meta-benchmark for evaluating cybersecurity AI agents," *arXiv:2510.24317*, 2025. [Online]. Available: https://arxiv.org/abs/2510.24317

[12] M. Shao *et al.*, "NYU CTF Bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2024.

[13] A. K. Zhang *et al.*, "Cybench: A framework for evaluating cybersecurity capabilities and risk of language models," *arXiv:2408.08926*, 2024.

[14] Ludus Cloud, "Ludus: Cyber range automation on Proxmox." [Online]. Available: https://ludus.cloud

[15] M3, "GOAD: Game of Active Directory." [Online]. Available: https://github.com/Orange-Cyberdefense/GOAD

[16] C. Phillips and L. P. Swiler, "A graph-based system for network-vulnerability analysis," in *Proc. New Security Paradigms Workshop*, 1998.

[17] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proc. IEEE Symp. Security and Privacy*, 2002.

[18] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in *Proc. 14th USENIX Security Symp. (USENIX Security 05)*, 2005, pp. 113–128. [Online]. Available: https://www.usenix.org/legacy/events/sec05/tech/full_papers/ou/ou.pdf

[19] OWASP Foundation, "OWASP Internet of Things," 2024. [Online]. Available: https://owasp.org/www-project-internet-of-things/

[20] MITRE Corporation, "MITRE ATT&CK for ICS," 2024. [Online]. Available: https://attack.mitre.org/matrices/ics/

[21] K. Stouffer *et al.*, "Guide to operational technology security," National Institute of Standards and Technology, NIST SP 800-82 Rev. 3, 2023.

[22] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Stratosphere Laboratory, 2020. [Online]. Available: https://www.stratosphereips.org/datasets-iot23

[23] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020.

[24] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.

[25] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, "A large-scale analysis of the security of embedded firmwares," in *Proc. 23rd USENIX Security Symp. (USENIX Security 14)*, 2014, pp. 95–110. [Online]. Available: https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/costin

[26] D. D. Chen, M. Egele, M. Woo, and D. Brumley, "Towards automated dynamic analysis for Linux-based embedded firmware," in *Proc. NDSS*, 2016.

[27] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, and Y. Kim, "FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis," in *Proc. Annual Computer Security Applications Conf. (ACSAC)*, 2020.

[28] OWASP, "IoTGoat: A deliberately insecure IoT firmware," 2024. [Online]. Available: https://github.com/OWASP/IoTGoat

[29] MiniMax, "MiniMax-M2.7: A general-purpose large language model," 2026. [Online]. Available: https://www.minimax.io/models/text/m27